1

Unit 1: Data Representation and Digital Fundamentals

Computer Organization and Architecture

CHANDRAS EDU

Never stop learning

Agenda

1.1 Data Types & Number Systems
1.2 Complements
1.3 Fixed-Point Representation
1.4 Floating-Point Representation
1.5 Binary Codes
1.6 Binary Storage & Registers
1.7 Binary Logic
1.8 Digital Logic Gates
1.9 K-Map Representation
1.10 Introduction to Digital Computers
1.11 Block Diagram of a Digital Computer
1.12 Computer Organization vs Architecture
1.13 Computer Design

1.1 Data Types & Number Systems

IN SIMPLE TERMS

Every number system works the same way: each digit is multiplied by the base raised to its position. Octal and hex are just short ways of writing groups of 3 or 4 bits.

What data does a computer store?

- Registers hold **control information** (bits that sequence the hardware's operations) or **data**
- Data falls into three categories: **numbers** for arithmetic, **letters** for text, and **other discrete symbols** (punctuation, control characters)
- Except for numbers already in pure binary, everything is stored in **binary-coded** form: an agreed mapping from bit patterns to symbols

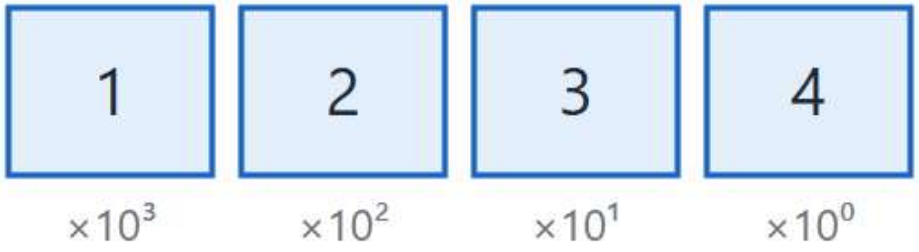
Common number systems

A number system of **radix** (base) r uses r distinct digit symbols, 0 through r − 1.

System	Radix (r)	Digit symbols	Used for
Decimal	10	0 – 9	Human input/output
Binary	2	0, 1	Hardware storage
Octal	8	0 – 7	Shorthand for binary
Hexadecimal	16	0–9, A–F	Compact binary shorthand

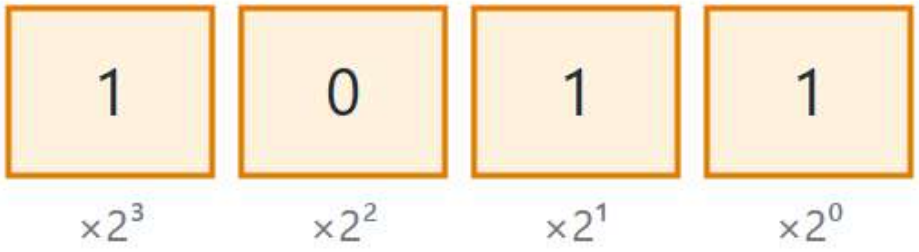
Positional notation

Decimal



$(1234)_{10}$
$= 1000 + 200 + 30 + 4$
$= 1234$

Binary



$(1011)_2$
$= 8 + 0 + 2 + 1$
$= (11)_{10}$

Each digit is weighted by the radix raised to its position

Positional notation — same rule, different radix.

Any radix → decimal

$$(101101)_2 = 1\times2^5+0\times2^4+1\times2^3+1\times2^2+0\times2^1+1\times2^0 = 32+8+4+1 = (45)_{10}$$

$$(736.4)_8 = 7\times8^2+3\times8^1+6\times8^0+4\times8^{-1} = 448+24+6+0.5 = (478.5)_{10}$$

$$(F3)_{16} = 15\times16^1+3\times16^0 = 240+3 = (243)_{10}$$

- Expand the positional-notation formula and add up the terms

Decimal → any radix

- **Integer part:** divide repeatedly by r; read the remainders from last to first (bottom to top)
- **Fractional part:** multiply repeatedly by r; read the carry digits from first to last (top to bottom)
- The fractional conversion may not terminate

Convert $(41.6875)_{10}$ to binary

$(41.6875)_{10} = (101001.1011)_2$

Integer ÷ 2	Remainder ↑	Fraction × 2	Carry ↓
41 ÷ 2 = 20	1	0.6875 × 2 = 1.375	1
20 ÷ 2 = 10	0	0.375 × 2 = 0.75	0
10 ÷ 2 = 5	0	0.75 × 2 = 1.5	1
5 ÷ 2 = 2	1	0.5 × 2 = 1.0	1
2 ÷ 2 = 1	0	(terminates)	
1 ÷ 2 = 0	1 ← MSB		

Convert $(41.6875)_{10}$ to octal and hexadecimal

Octal:

Integer: 41 ÷ 8 = 5 rem 1; 5 ÷ 8 = 0 rem 5 → integer = 51

Fraction: 0.6875 × 8 = 5.5 → 5; 0.5 × 8 = 4.0 → 4

Result: $(51.54)_8$

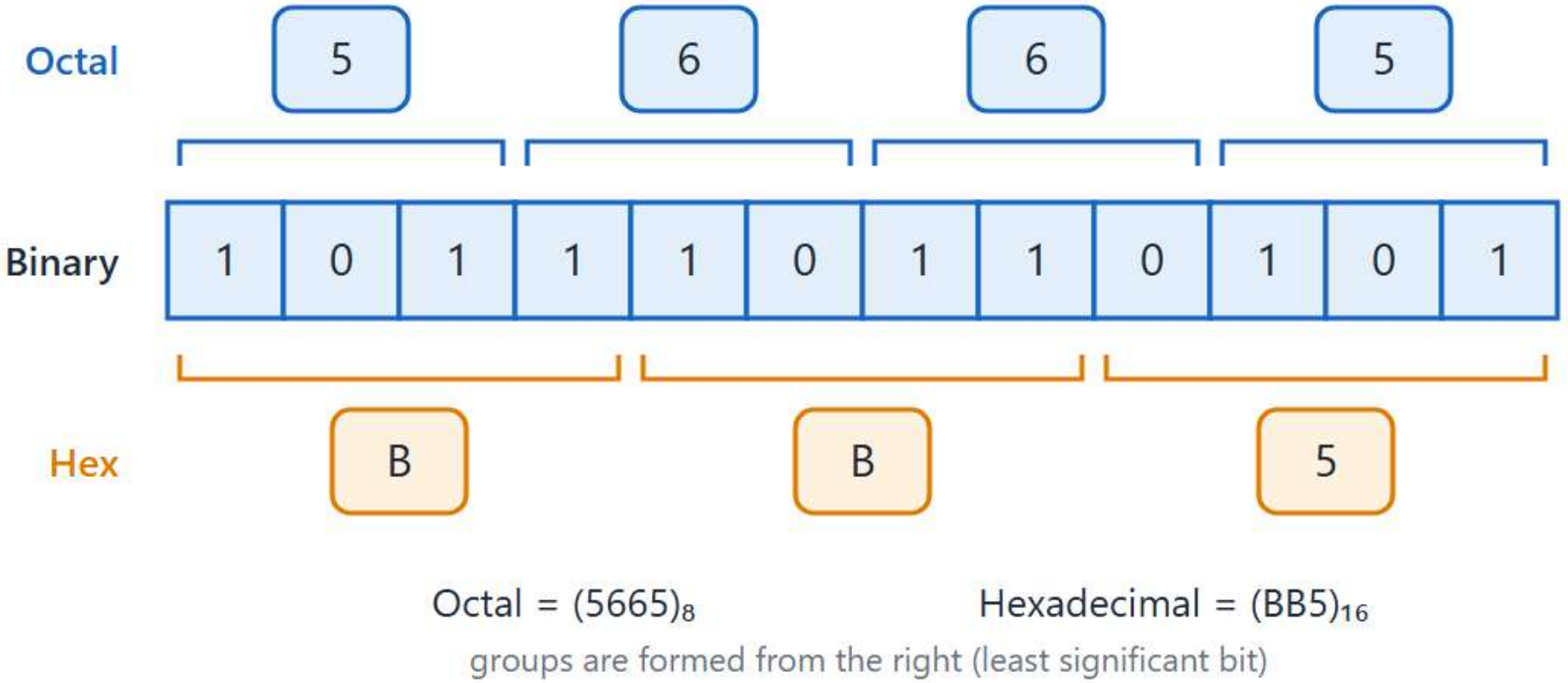
Hexadecimal:

Integer: 41 ÷ 16 = 2 rem 9; 2 ÷ 16 = 0 rem 2 → integer = 29

Fraction: 0.6875 × 16 = 11.0 → B (terminates)

Result: $(29.B)_{16}$

Bit grouping: binary ↔ octal and hexadecimal



Bit grouping — binary ↔ octal (3 bits) and hexadecimal (4 bits).

1.2 Complements

IN SIMPLE TERMS

A complement lets a computer subtract by adding: add the complement of the number you want to take away, then look at the end carry.

What are complements?

- A complement is defined relative to the **radix r** and a fixed number of digits **n**
- Every radix has two: the **(r – 1)'s complement** and the **r's complement**
- **(r – 1)'s complement of N = $(r^n - 1) - N$**
- **r's complement of N = $r^n - N$** = (r − 1)'s complement + 1
- The second formula is the first plus 1: the key to computing either quickly

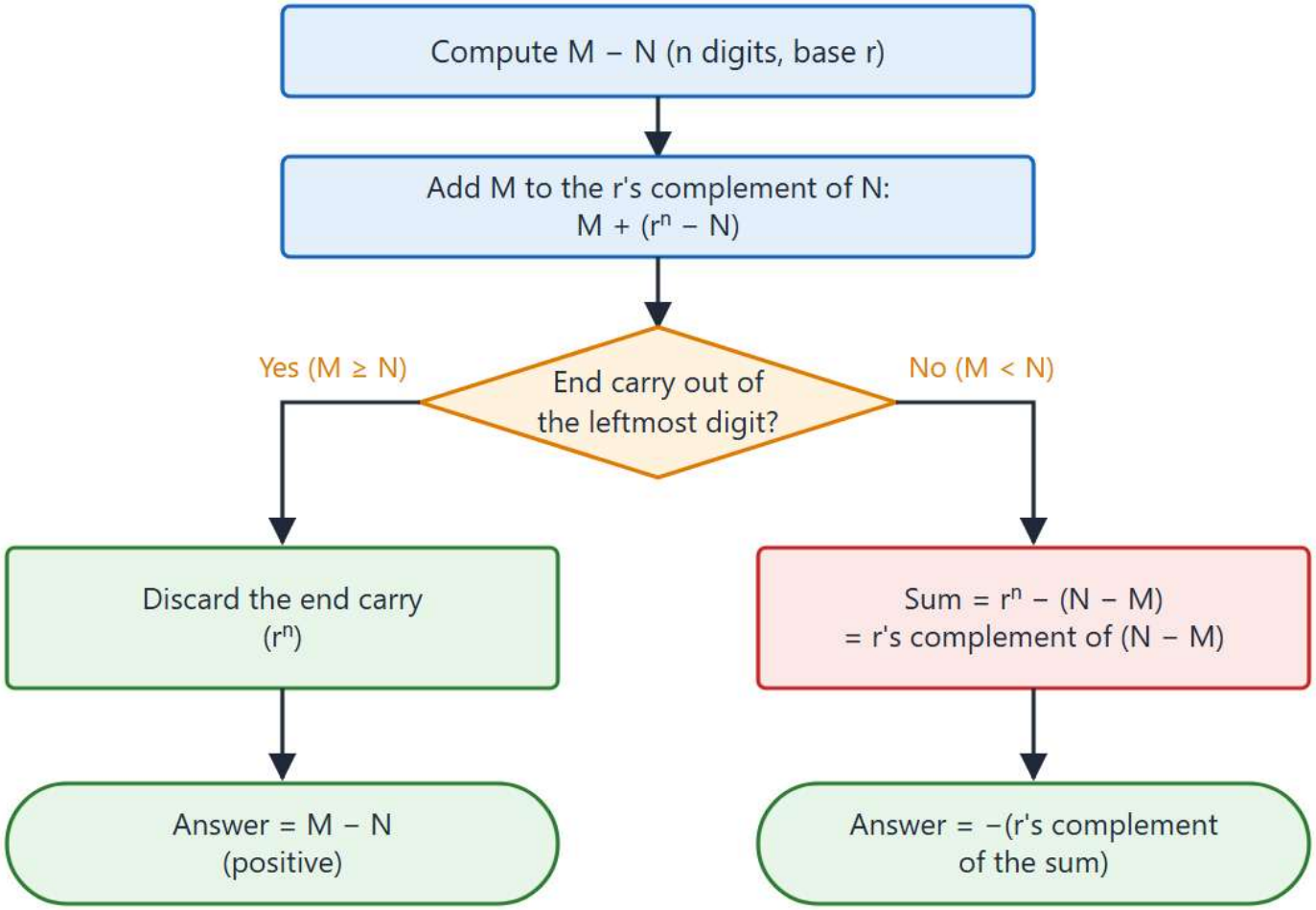
Complements in decimal and binary

- **Decimal 9's complement:** subtract each digit from 9, or $(10^n - 1) - N$
- **Decimal 10's complement:** the 9's complement plus 1
- **Binary 1's complement:** flip every bit: just inverters, the fastest complement in hardware
- **Binary 2's complement:** the 1's complement plus 1; used universally for signed integers

Examples: finding complements


```
Decimal, N = 546700:
9’s complement  = 999999 – 546700 = 453299
10’s complement = 453299 + 1      = 453300
Binary, N = 1011001:
1’s complement: flip every bit → 0100110
2’s complement: 0100110 + 1    → 0100111
```

Subtraction with the r’s complement



Subtraction of unsigned numbers using the r's complement: the end carry decides the sign of the result.

The end carry decides the sign

End carry?	Meaning	Action
✅ Yes	M ≥ N → positive result	Discard the carry. Result is M − N.
❌ No	M < N → negative result	Take the r's complement of the sum and attach a minus sign.

Example 1: decimal, carry produced (M ≥ N)


```
72532 – 13250:
  10's complement of 13250 = 86750
  Sum = 72532 + 86750      = 159282
  End carry → discard it   → 59282  ✓ (positive)
```

Example 2: decimal, no carry (M < N)


```
13250 – 72532:
  10’s complement of 72532 = 27468
  Sum = 13250 + 27468      = 40718
  No end carry → negative
  10’s complement of 40718 = 59282
  Answer = –59282  ✓
```

Example 3: binary, X = 1010100 (84), Y = 1000011 (67)


```
X − Y:
  2's complement of Y = 0111101
  Sum = 1010100 + 0111101 = 10010001
  End carry → discard → 0010001 = +17  ✓
Y − X:
  2's complement of X = 0101100
  Sum = 1000011 + 0101100 = 1101111
  No end carry → take 2's complement → 0010001
  Answer = −17  ✓
```

1.3 Fixed-Point Representation

IN SIMPLE TERMS

A computer has no minus sign, so the leftmost bit becomes the sign. Two’s complement is used because adding and subtracting then work with one simple adder.

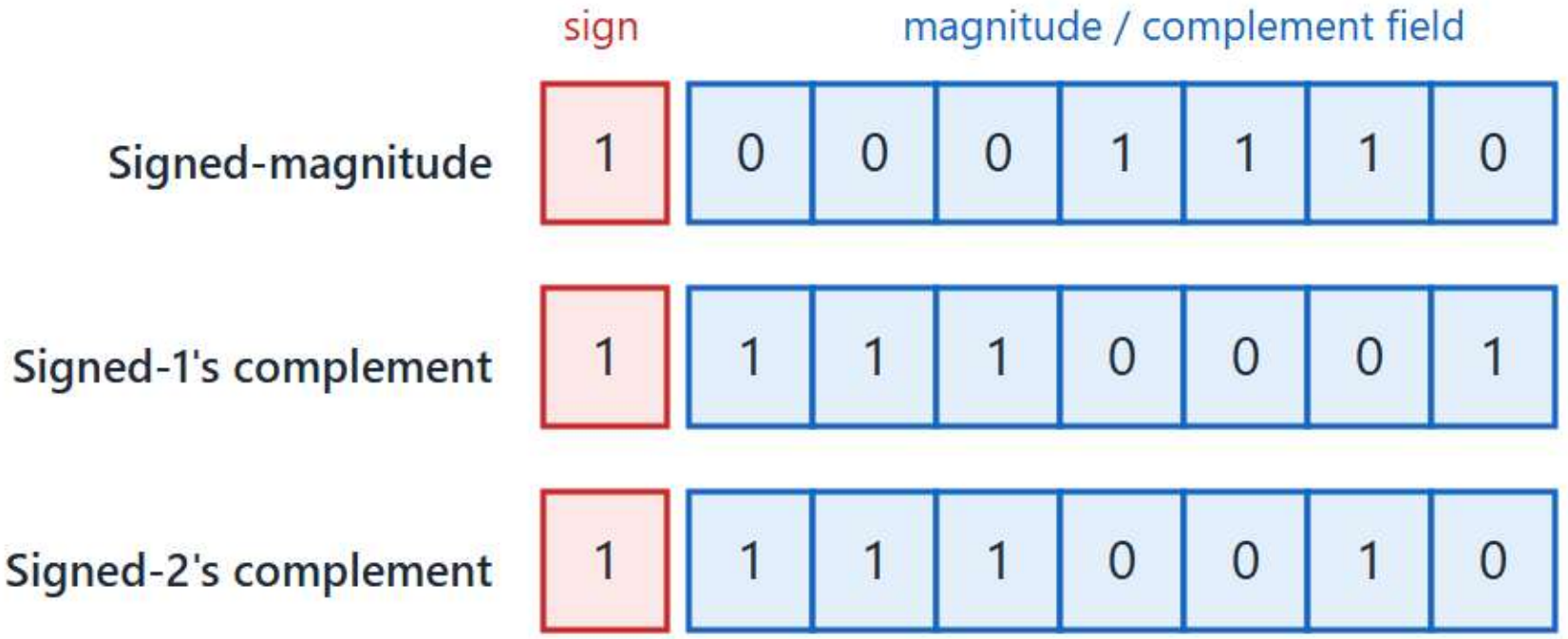
No minus sign in hardware

- The leftmost bit is a **sign flag**: 0 means positive, 1 means negative
- In an 8-bit number that leaves 7 bits for the value
- **Fixed-point:** everyone agrees in advance where the binary point goes, and it never moves
- Point at the far left → every number is a fraction; point at the far right → every number is a whole number
- This course uses whole numbers; the point is never stored as a bit

Three ways to write a negative number

- All three agree on positive numbers; they differ only when negative
- **Signed-magnitude:** keep ordinary binary and flip only the sign bit to 1
- **Signed-1’s-complement:** flip **every** bit of the positive version
- **Signed-2’s-complement:** take the 1’s complement, then add 1

–14 in an 8-bit register



+14 = 0 0001110 in every convention

–14 in an 8-bit register under all three conventions (compare +14 = 00001110).

Writing –20, –50 and –100 in 8 bits

Steps for −20: 1) write +20 as 00010100 2) signed-magnitude: flip only the sign bit → 10010100
3) 1’s complement: flip every bit → 11101011 4) 2’s complement: add 1 → 11101100

Number	Positive version	Signed-magnitude	1’s complement	2’s complement
−20	00010100	10010100	11101011	11101100
−50	00110010	10110010	11001101	11001110
−100	01100100	11100100	10011011	10011100

Adding signed numbers

- **Signed-magnitude:** same signs → add magnitudes, keep the sign; different signs → subtract the smaller from the larger, keep the bigger's sign
- The hardware needs a comparator **and** a subtractor, on top of an adder
- **2's complement:** add the two numbers, sign bits included, like normal addition
- If a carry pops out of the leftmost position, throw it away
- One rule, one piece of hardware: the reason 2's complement is used in every real computer

2’s complement addition (8 bits)

What we are adding	In 8-bit binary	Result	Matches normal math?
(+6) + (+13)	00000110 + 00001101	00010011	= +19 ✓
(−6) + (+13)	11111010 + 00001101	00000111	= +7 ✓
(+6) + (−13)	00000110 + 11110011	11111001	= −7 ✓
(−6) + (−13)	11111010 + 11110011	11101101 (carry dropped)	= −19 ✓

Example: (+30) + (−12)


```
+30   =   00011110
−12   =   11110100
---------------------
sum   =   100010010     ← 9 bits! drop the leading carry
keep  =   00010010      =   +18
```

- Check: 30 − 12 = 18. It matches, using nothing but plain addition

Overflow: when the answer does not fit

- An 8-bit signed register holds −128 to +127; an answer outside that range is **overflow**
- **Unsigned:** overflow shows as an extra carry out of the last bit
- **Signed:** overflow happens exactly when the carry **into** the sign bit differs from the carry **out of** it
- Positive + negative can **never** overflow; only two positives or two negatives can
- Using the unsigned carry trick on signed numbers gives the wrong answer

Example: overflow, (+70) + (+80)


```
+70   =   01000110
+80   =   01010000
--------------------
sum   =   10010110   ← looks negative!
```

- It looks negative, but we added two positives: this signals overflow
- 150 does not fit in 8 signed bits (maximum is +127)

1.4 Floating-Point Representation

IN SIMPLE TERMS

Floating point is scientific notation in binary: a fraction (the digits) and an exponent (where the point goes), so very large and very small numbers fit in the same register.

Why not fixed-point? Mantissa and exponent

- Fixed-point: a huge number needs many bits before the point; a tiny precise fraction needs many after; a fixed register cannot give both
- Floating point lets the point **float**, and stores where it moved to
- **Mantissa (fraction):** the meaningful digits, as a signed value
- **Exponent:** a power of the base (2 for binary): how far and which way to slide the point
- **value = mantissa × $(\text{base})^{\text{exponent}}$**
- Only the mantissa and exponent are stored; the base and the point position are conventions

Examples 1 and 2

Example 1: $+6132.789 = 0.6132789 \times 10^{+4}$

Mantissa = +0.6132789, Exponent = +04

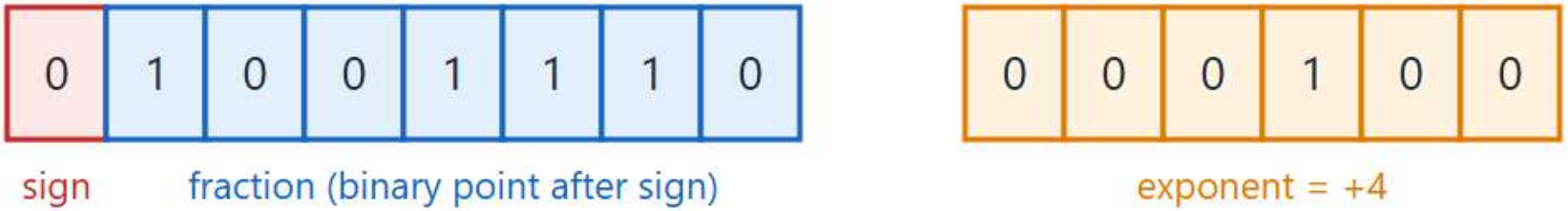
Example 2: binary +1001.11, 8-bit fraction, 6-bit exponent

Mantissa = 01001110, Exponent = 000100

representing $+(.1001110)_2 \times 2^{+4}$

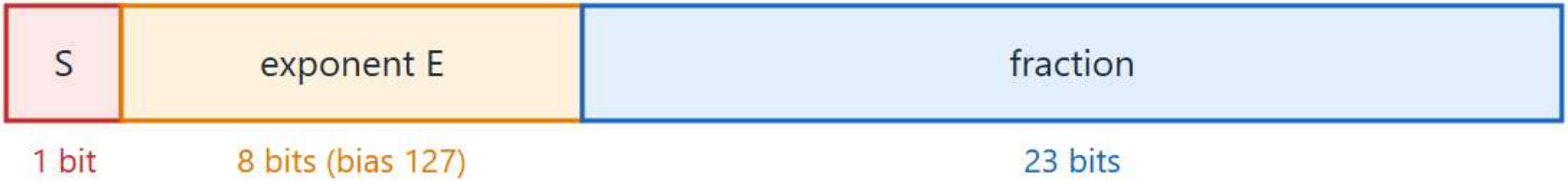
Classic floating-point layout vs. IEEE 754 single precision

Textbook format — example +1001.11



$$\text{value} = m \times 2^{e} = +(.1001110)_2 \times 2^{+4}$$

IEEE 754 single precision (for comparison)



$$\text{value} = (-1)^{s} \times 1.\text{fraction} \times 2^{(E\,-\,127)}$$

Classic floating-point layout vs. IEEE 754 single precision.

Example 3: converting +37.25 step by step


```
Step 1: 37.25 in binary is 100101.01
Step 2: slide the point left until only a 1 is before it:
        100101.01 → 0.10010101  (slid 6 places)
Step 3: Mantissa = 10010101, Exponent = +6 (000110)
```

- Check: $0.10010101_2 = 0.58203125$, and $0.58203125 \times 2^6 (= 64) = 37.25$ ✓

Normalization and zero

- A number is **normalized** when the first digit of the mantissa is 1, not 0
- This squeezes the most precision out of a fixed-size mantissa: no bits wasted on leading zeros
- **Rule:** however many places you slide the mantissa left, subtract the same number from the exponent
- Example A: 00011010 has 3 leading zeros → shift left 3 → 11010000; exponent reduced by 3
- Example B: 00000101 has 5 leading zeros → shift left 5 → 10100000; exponent reduced by 5
- **Zero** cannot be normalized: it is stored as all-zero mantissa and all-zero exponent

1.5 Binary Codes

IN SIMPLE TERMS

A binary code just gives each symbol a pattern of bits. BCD codes decimal digits, ASCII codes characters, and Gray code changes only one bit between neighbours.

Binary codes and BCD

- A binary code assigns a group of n bits (up to 2^n patterns) to represent a digit, character or position
- A code need not use every pattern
- **BCD (8421):** each decimal digit 0–9 is coded separately in 4 bits, with weights 8, 4, 2, 1
- Six patterns (1010–1111) are **unused** in BCD
- BCD is **not** the same as converting the whole decimal number to binary

BCD versus straight binary

Decimal	BCD (digit by digit)	Straight binary (whole number)
396	0011 1001 0110	110001100
99	1001 1001	1100011

Self-complementing codes: Excess-3


```
Digit 6 → BCD 0110 → Excess-3 = 0110 + 0011 = 1001
Flip every bit of 1001 → 0110
Decode as Excess-3 (subtract 3): 0110 − 0011 = 0011 = digit 3
```

- **Self-complementing:** the 9’s complement of a digit is found just by flipping every bit
- **Excess-3:** the BCD value plus 3; the **2421** code (weights 2, 4, 2, 1) is also self-complementing
- Check: the 9’s complement of 6 is 9 − 6 = 3 ✓

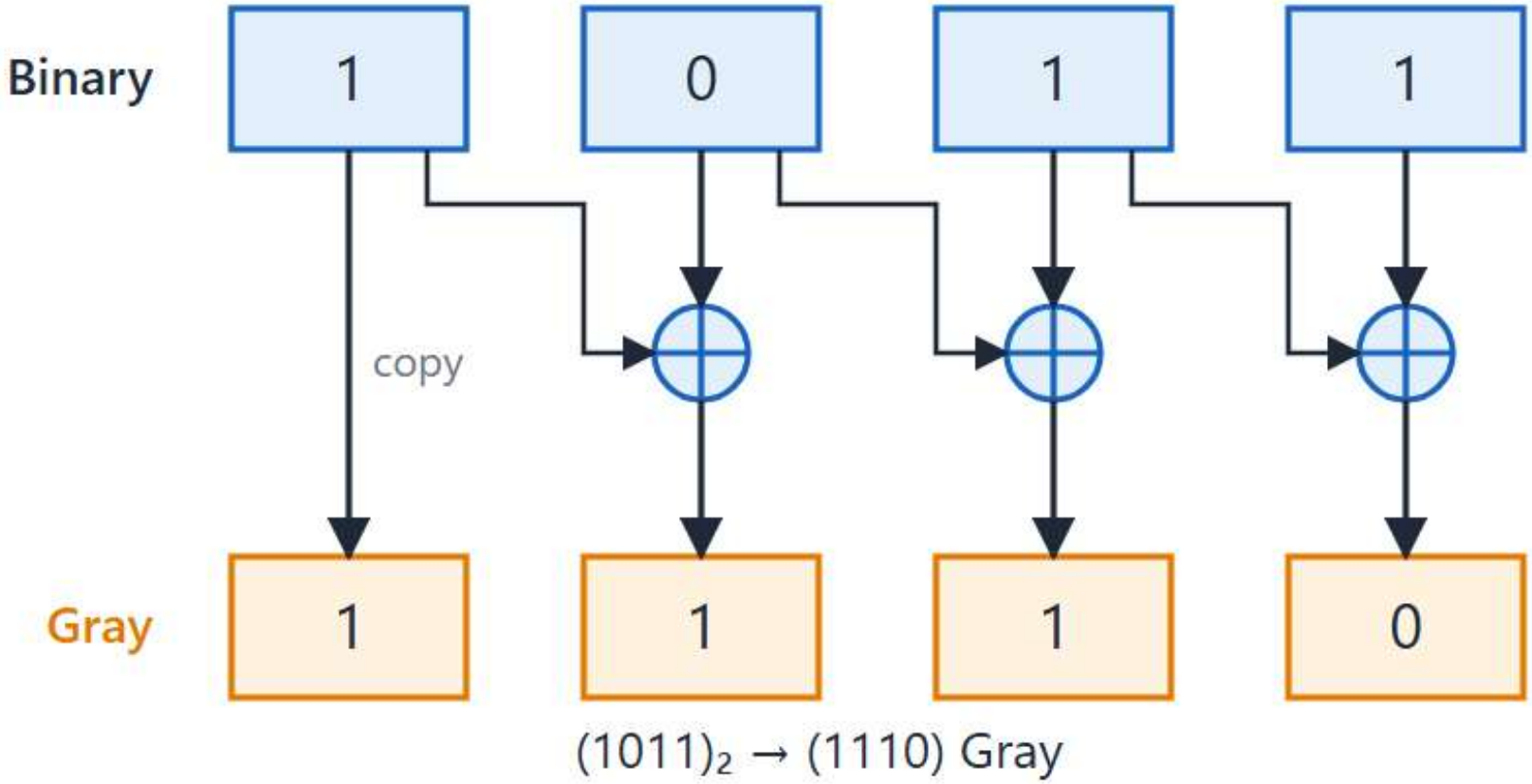
Gray code

- **Gray code** (reflected binary, unit-distance code): consecutive values differ in exactly **one** bit
- Ordinary binary does not: 0111 → 1000 flips all four bits
- Bits can change at slightly different instants; in binary this can give a wildly wrong intermediate value
- In Gray code at most one bit is mid-transition: the worst case is one step off
- Used in rotary position encoders (motor controllers, robotics, old mechanical mice)

Binary and Gray code, 0 to 7

Decimal	Binary	Gray code
0	0000	0000
1	0001	0001
2	0010	0011
3	0011	0010
4	0100	0110
5	0101	0111
6	0110	0101
7	0111	0100

Binary-to-Gray conversion



Binary-to-Gray conversion — MSB copied, every other bit is an XOR with its neighbor.

ASCII and parity

- **ASCII** is the standard alphanumeric code: 7 bits, 128 characters (letters, digits, punctuation, control characters)
- A **parity bit** is one extra bit appended so the total count of 1s follows a fixed rule
- **Even parity:** the total is even; **odd parity:** the total is odd
- It catches a single flipped bit but cannot say **which** bit flipped
- It misses an even number of simultaneous flips

1.6 Binary Storage & Registers

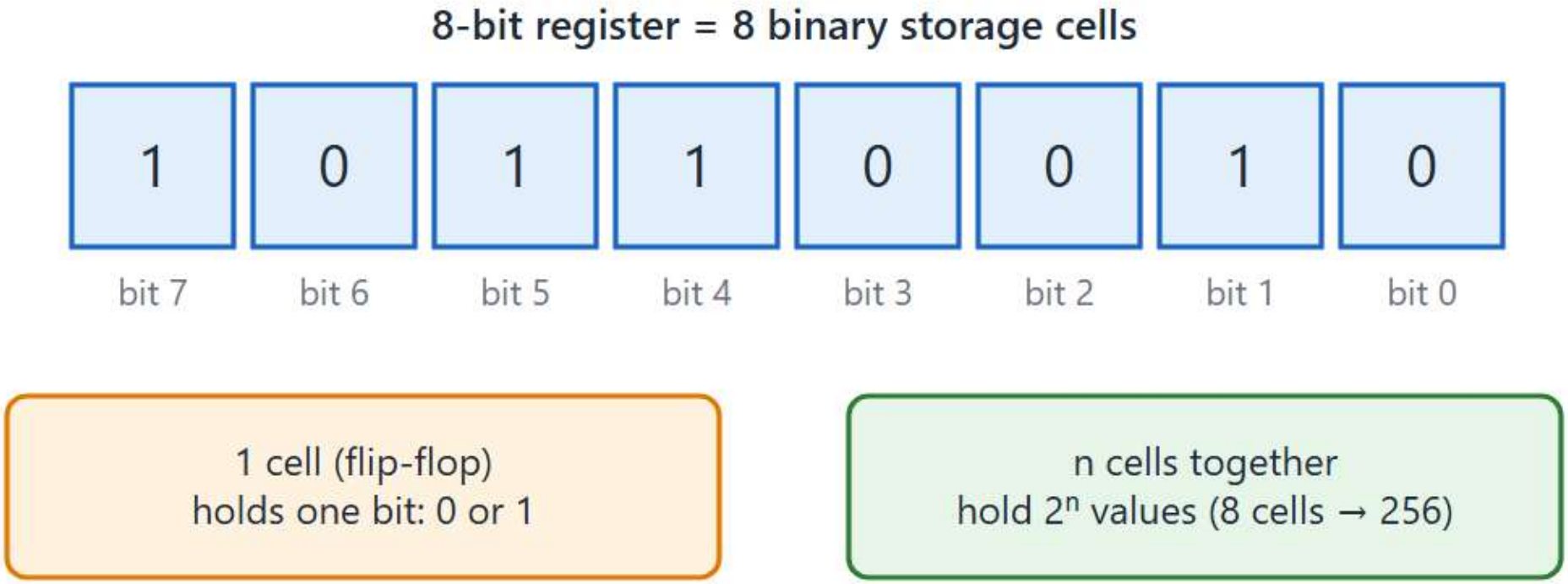
IN SIMPLE TERMS

A register is a row of one-bit storage cells; n cells can hold any one of 2^n values.

The storage cell

- A **binary storage cell** is a **flip-flop**: a circuit with exactly two stable electrical states
- The state it sits in **is** the bit: one state means 0 and the other means 1
- It **holds** its state without continuous input, unlike a plain wire
- Treat it as a labelled box that stores one bit; how it is built comes in the Sequential Circuits unit

A register of storage cells



A register is a group of storage cells; n cells hold one of 2^n possible values.

Registers and register transfer

- A **register** is an ordered group of storage cells acting together; an 8-bit register is 8 flip-flops side by side
- n cells hold any one of $\mathbf{2^n}$ values: $2^8 = 256$; 2^{32} is over 4 billion
- A **register transfer** moves or copies a value from one register to another
- A transfer does not necessarily destroy the original

1.7 Binary Logic

IN SIMPLE TERMS

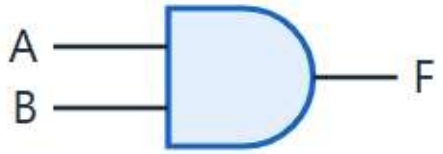
Binary logic has just three basic operations: AND (all must be 1), OR (any can be 1) and NOT (flip it).

Binary variables and three operations

- A **binary variable** takes exactly one of two values: 0 or 1
- In binary logic read them as **truth values**: 1 = true, 0 = false
- **AND** (A·B or AB): 1 only when **both** A and B are 1
- **OR** (A+B): 1 when **at least one** of A or B is 1
- **NOT** (A’ or $\bar{A}$): flips the value: 1 becomes 0 and 0 becomes 1

Three basic operations and truth tables

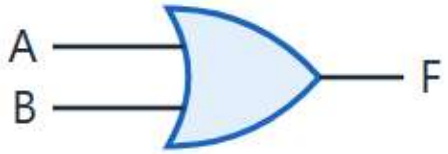
AND



$F = A \cdot B$

A	B	F
0	0	0
0	1	0
1	0	0
1	1	1

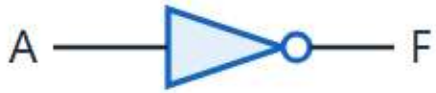
OR



$F = A + B$

A	B	F
0	0	0
0	1	1
1	0	1
1	1	1

NOT



$F = A'$

A	F
0	1
1	0

The three basic binary logic operations and their truth tables.

Gates, and a warning about symbols

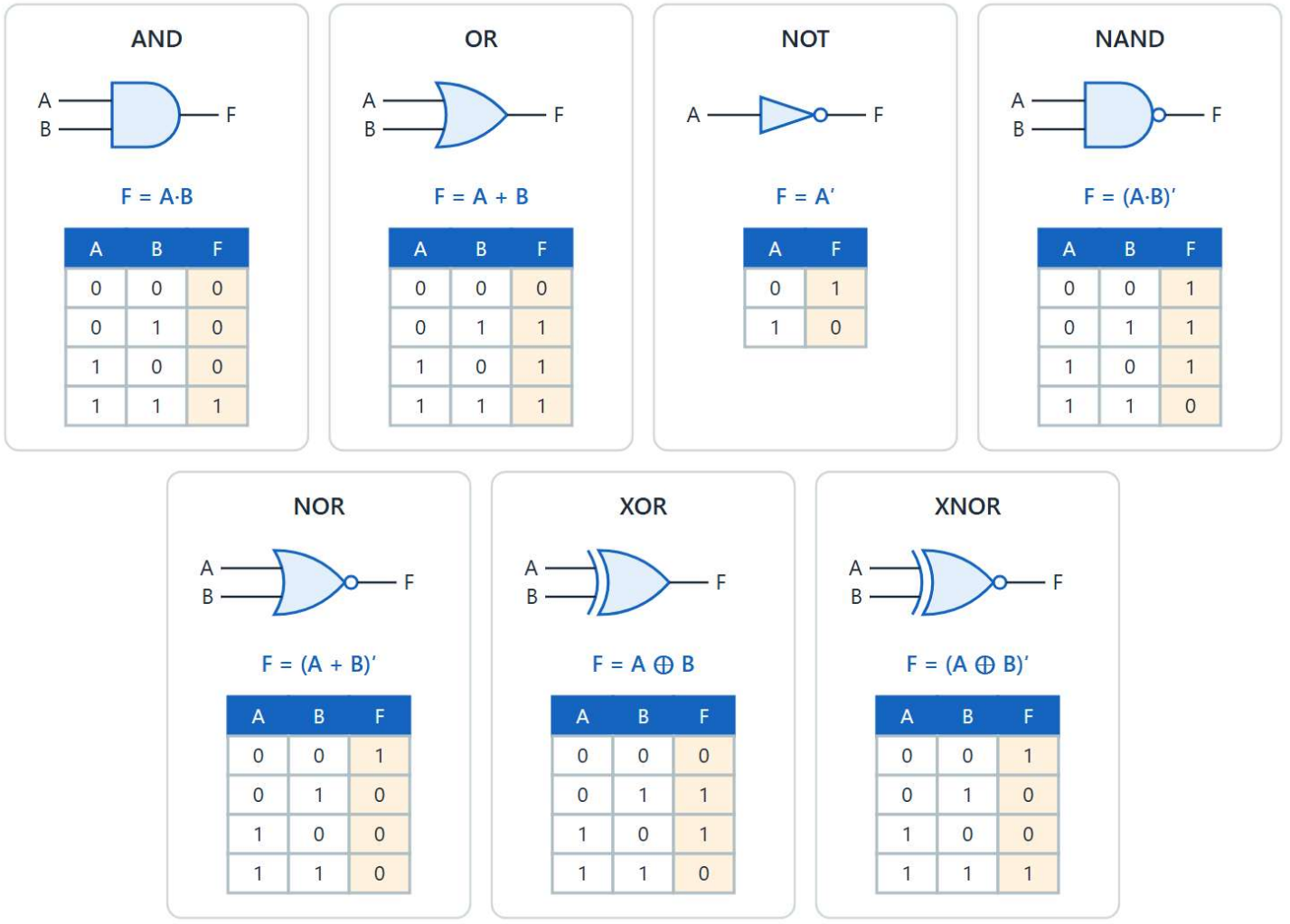
- A **logic gate** is the physical circuit that carries out one of these operations electronically
- AND, OR and NOT map onto the AND gate, the OR gate and the NOT gate (inverter)
- **Warning:** + does not mean addition and · does not mean multiplication here
- In binary logic 1 + 1 = 1 (OR), not 2

1.8 Digital Logic Gates

IN SIMPLE TERMS

Each gate is a tiny circuit for one logic operation; NAND or NOR alone can build every other gate.

The seven basic gates



Symbol, expression, and truth table for each of the seven basic gates.

DeMorgan’s theorem

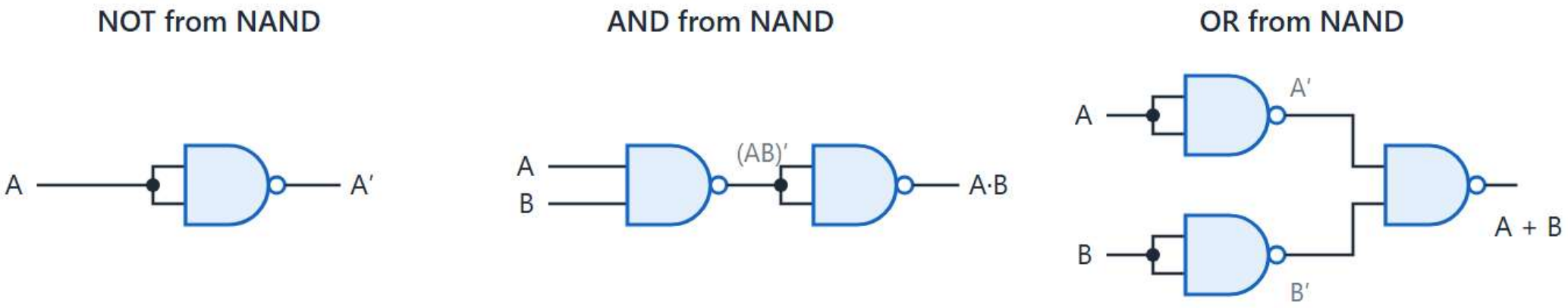
$$(A + B)' = A' \cdot B'$$
$$(A \cdot B)' = A' + B'$$

- Complementing an expression flips the operator (+ becomes ·, and vice versa) **and** complements every literal
- Check for A = 0, B = 1: (A+B)’ = (0+1)’ = 1’ = 0 and A’·B’ = 1·0 = 0: match
- True for all 4 input combinations, which is what makes it a theorem

NAND and NOR are universal gates

- Either one alone can build **every** other gate, NOT, AND and OR included
- Real chips are built almost entirely from NAND (or NOR): one gate type made repeatedly is far cheaper
- **NOT from NAND:** tie both inputs together: NAND(A,A) = (A·A)′ = A′
- **AND from NAND:** a NAND followed by a second NAND wired as a NOT
- **OR from NAND:** invert both inputs, then NAND: (A′·B′)′ = A + B

NOT, AND and OR from NAND gates



Any Boolean function can be built from NAND gates alone — NAND is a universal gate

NOT, AND, and OR, each built using only NAND gates.

Canonical form: minterms


```
F(A,B,C) is 1 for rows 1, 3, 5, 6  →  F = Σm(1,3,5,6)
F = A'B'C + A'BC + AB'C + ABC'
```

- A **minterm** is an AND term containing every variable exactly once: uncomplemented if the variable is 1 in that row, complemented if 0
- Writing a function as the OR of all its minterms is the **canonical sum-of-products (SOP)** form
- Guaranteed correct but rarely minimal: the starting point for Karnaugh maps

1.9 K-Map Representation

IN SIMPLE TERMS

A K-map lays out the truth table so that neighbouring cells differ in one bit; circling neighbouring 1s in groups of 1, 2, 4 or 8 gives the simplest expression.

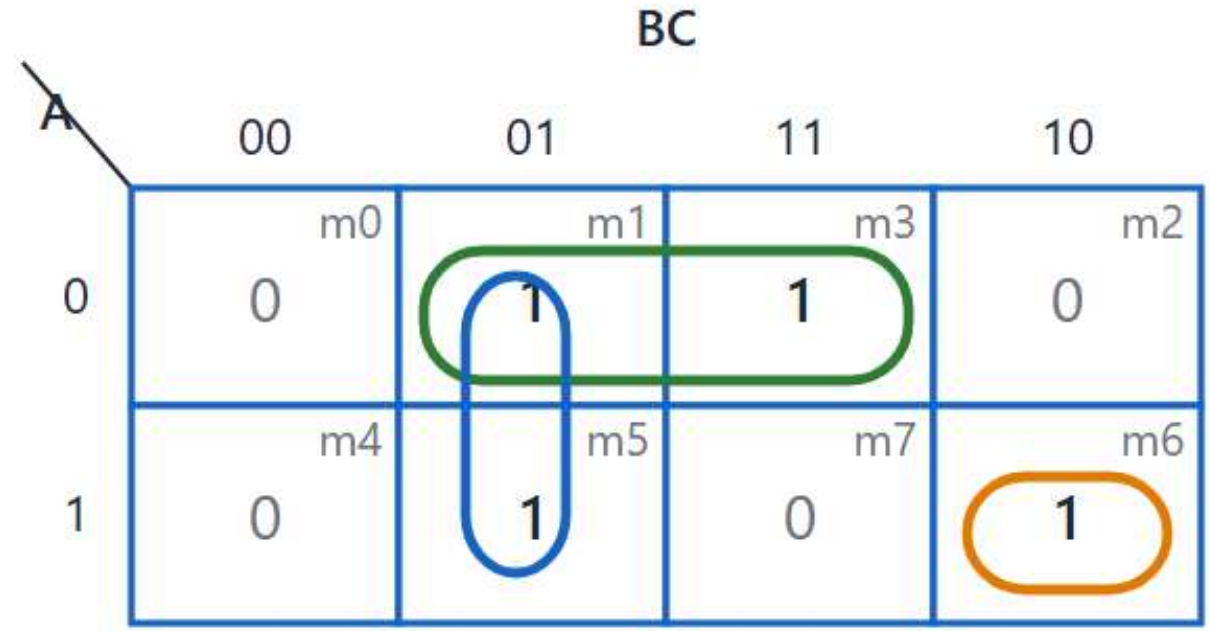
Why a grid works

- A K-map places truth-table rows in a grid so physically adjacent cells differ in **exactly one** variable
- This reuses **Gray code**: headers are written 00, 01, 11, 10, never 00, 01, 10, 11
- Neighbours are one bit-flip apart across, down, and even wrapping around the edges
- Two cells that differ in one variable combine algebraically: the differing variable cancels
- A K-map lets you **see** which cells qualify

Grouping rules

- Group adjacent 1s in powers of two: 1, 2, 4, 8, … cells
- Groups must be rectangular, including **wraparound** across top/bottom and left/right edges
- Bigger groups are better: doubling a group removes one more variable from its term
- Every 1 must be covered by at least one group, using as few groups as possible
- **Don’t-care cells (X)** may join a group only if that makes the group larger

Three-variable K-map



A′C (m1, m3) B′C (m1, m5) ABC′ (m6)

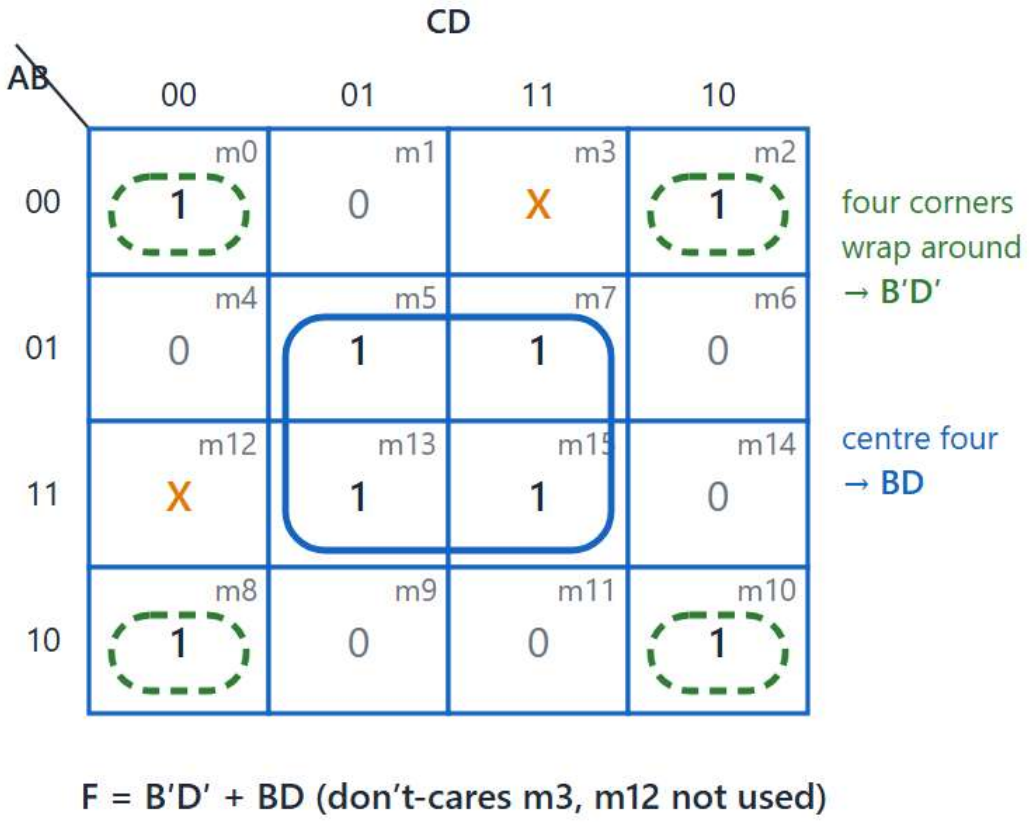
$$F = A'C + B'C + ABC'$$

Three-variable K-map for F = Σm(1,3,5,6): F = A′C + B′C + ABC′.

Worked example: 3 variables

- F(A,B,C) = Σm(1,3,5,6), the function from the previous topic
- m1 and m3 (adjacent across BC = 01/11 in row A = 0): B differs and cancels → **A’C**
- m5 pairs with m1 (same column, BC = 01) → **B’C**
- m6 has no neighbouring 1 → it stays alone as **ABC’**
- Minimal result: **F = A’C + B’C + ABC’**: three terms with fewer literals than the four-term canonical SOP

Four-variable K-map with don’t-cares



F = B’D’ + BD (don’t-cares m3, m12 not used)

Four-variable K-map: the four corners wrap around to give B’D’, the centre four give BD; the don't-cares m3 and m12 are not needed.

Worked example: 4 variables with don’t-cares

- The four corner cells (m0, m2, m8, m10) are mutually adjacent through wraparound
- Left edge connects to right edge, and top edge to bottom edge
- Grouped together they simplify to the single term **B’D’**
- The centre four 1s (m5, m7, m13, m15) form a second group: **BD**
- Neither don’t-care enlarges any group, so both are left out
- **F = B’D’ + BD**

1.10 Introduction to Digital Computers

IN SIMPLE TERMS

A digital computer works with just two values, 0 and 1; groups of bits stand for numbers, letters and instructions.

What “digital” means, and why binary

- A **digital computer** is a digital system that performs computational tasks
- **Digital:** information is represented by variables with a **limited number of discrete values**, e.g. the decimal digits 0–9
- The earliest digital computers (late 1940s) were used mainly for numerical computation
- Two states are more reliable than ten, and human logic is binary (true/false): components with two values are called **binary**
- A single binary digit is a **bit**; information is stored in groups of bits

The same bits can mean different things

Binary 1001011 as a number:

$$1\times2^{6} + 0\times2^{5} + 0\times2^{4} + 1\times2^{3} + 0\times2^{2} + 1\times2^{1} + 1\times2^{0}$$

$$= 64 + 0 + 0 + 8 + 0 + 2 + 1 = 75$$

- Those same seven bits could just as easily represent the letter “K” under a binary code for the alphabet
- Meaning comes entirely from the coding scheme applied to the bits

Hardware and software

- **Hardware:** all the electronic components and electromechanical devices that make up the physical machine
- **Software:** the instructions and data the computer manipulates for data processing
- A sequence of instructions is a **program**; the data it manipulates is its **data base**
- **System software** (the operating system) helps make more effective use of the computer; e.g. a compiler
- **Application programs** are written by users to solve their own problems
- A buyer needs both: hardware alone is useless without system software

1.11 Block Diagram of a Digital Computer

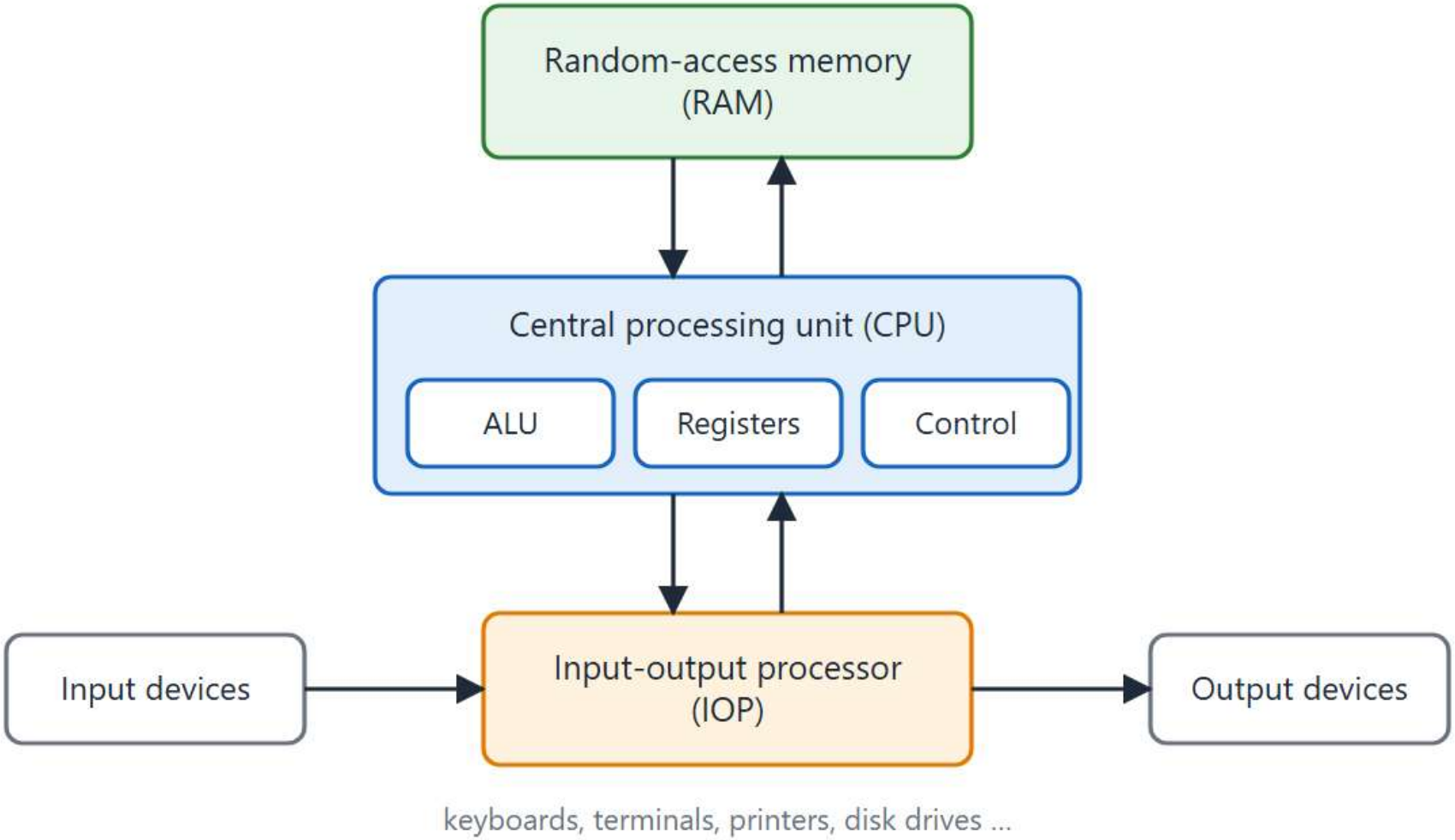
IN SIMPLE TERMS

Every computer has three parts: memory to hold programs and data, a CPU to process them, and an input-output processor to talk to the outside world.

The three major hardware parts

- **Central Processing Unit (CPU):** an arithmetic and logic unit, registers for storing data, and control circuits for fetching and executing instructions, all in one package
- **Memory:** storage for instructions and data
- **Input-Output Processor (IOP):** circuits for communicating with and controlling transfers to the outside world
- Keyboards, printers, terminals and disk drives connect to the computer through the IOP

Block diagram of a digital computer



Block diagram of a digital computer. Adapted from M. Morris Mano, Computer System Architecture, Fig. 1-1.

Inside the CPU, and why “random-access”?

- **ALU:** manipulates data with the arithmetic and logic operations built from logic gates
- **Registers:** small, extremely fast storage inside the CPU, for data being worked on
- **Control circuits:** fetch instructions from memory and execute them in the correct sequence
- **RAM:** the CPU can access any location at random in a fixed interval of time, whichever location it is
- The name is about the **access pattern**, not random data; contrast with sequential tape

1.12 Computer Organization vs Architecture

IN SIMPLE TERMS

Architecture is what the programmer sees (the instruction set); organization is how the hardware actually carries it out.

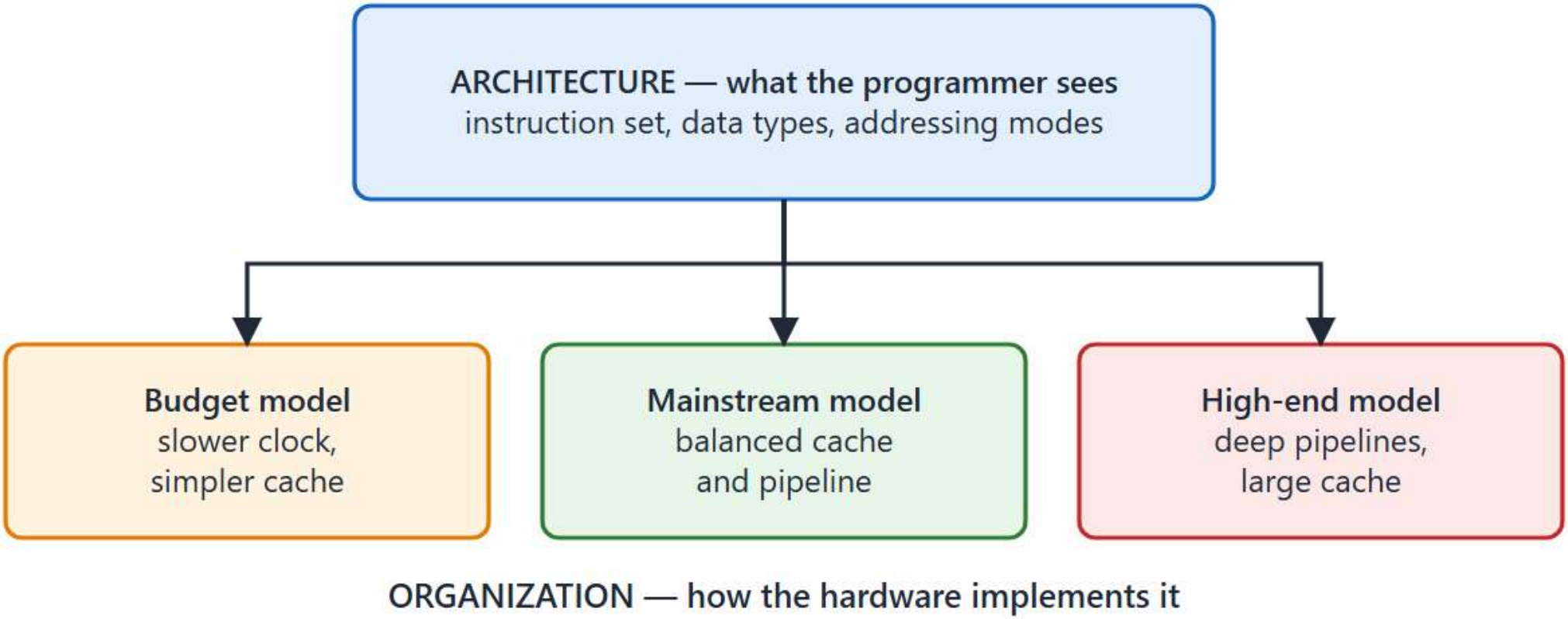
Three perspectives on computer hardware

- **Computer organization:** how the hardware components operate and are connected; the components are assumed already in place, and the task is to verify that they operate as intended
- **Computer design:** the hardware design; once specifications exist, determine what hardware to use and how to connect it (also called **computer implementation**)
- **Computer architecture:** the structure and behaviour of the computer **as seen by the user**: information formats, the instruction set, and addressing techniques

Architecture: the “what”

- The **instruction set:** the operations the machine understands (add, load, jump, compare…)
- **Information formats:** the data types the hardware directly supports
- **Addressing modes:** the techniques for addressing memory
- Architecture is a **specification**: a contract
- Two machines with the same architecture run the same programs, even if built completely differently inside

Same architecture, many organizations



The same architecture (instruction set) can be realized by many different organizations (hardware implementations) — a common illustration beyond the textbook's own examples.

Why the distinction matters

- Every chip implementing the same instruction set shares the **same architecture**
- Different models have different **organizations**: clock speeds, cache sizes, numbers of execution units
- **Software portability:** architecture is a stable contract, so software keeps working across hardware generations
- **Hardware innovation:** organization can change without breaking the architecture
- **Careers:** architecture work and organization work are different specializations

Architecture, organization and design compared

	Architecture	Organization	Design
Question answered	What does the machine do?	Does the built hardware operate as intended?	What hardware should be built, and how connected?
Visible to	The programmer (user)	The hardware engineer verifying it	The hardware designer building it
Mindset	Specification / contract	Verification of components already in place	Construction from a specification
Includes	Instruction set, information formats, addressing modes	Control signals, interfaces, memory technology	Covered in the next topic

1.13 Computer Design

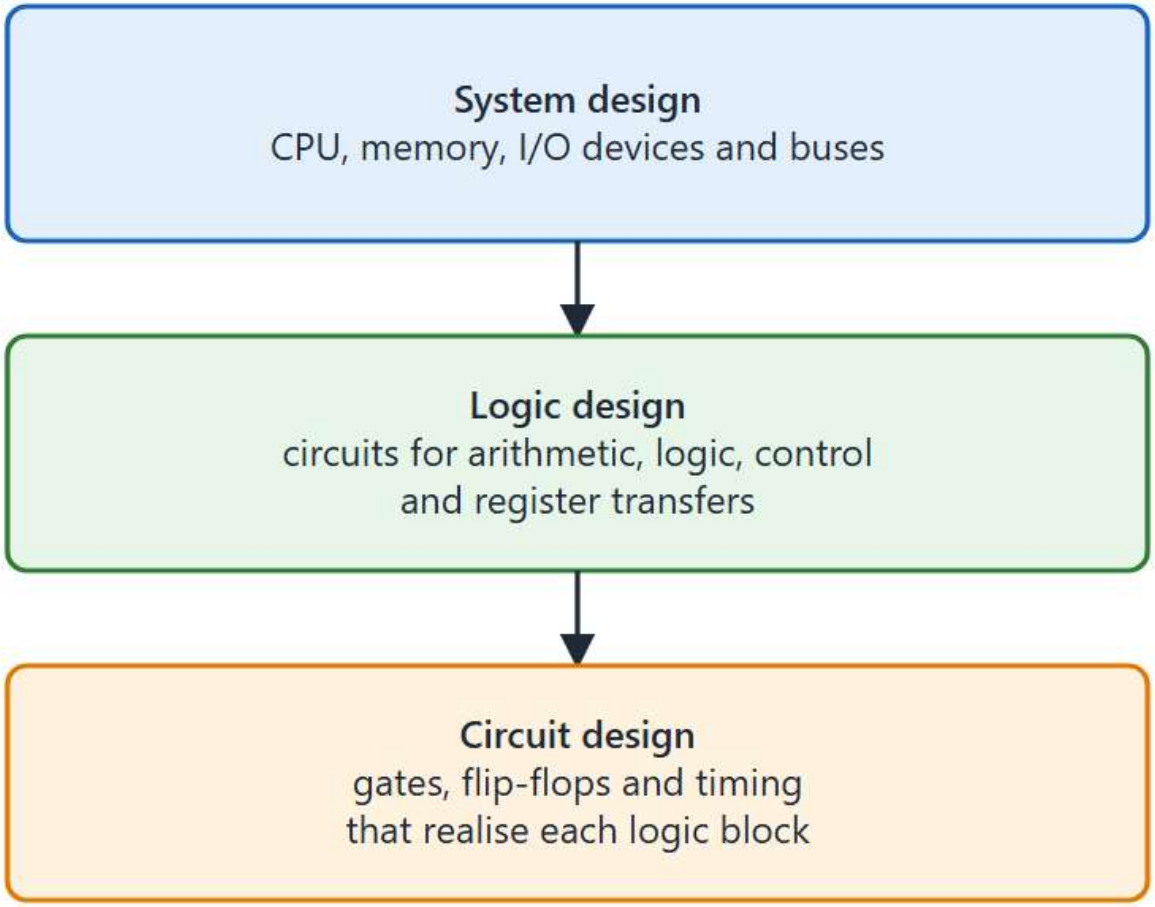
IN SIMPLE TERMS

Computer design goes from the big picture to the details: system blocks first, then the logic inside them, then the circuits.

Computer design

- Computer design is concerned with the hardware design of the computer
- Once specifications are formulated, the designer develops the hardware
- It determines **what hardware should be used** and **how the parts should be connected**
- Also called **computer implementation**
- Organization examines existing hardware; design starts from a specification and builds hardware to satisfy it

Three levels of design detail



- **System design:** how CPU, memory, I/O and buses fit together
- **Logic design:** circuits for arithmetic, logic, control and register transfers
- **Circuit design:** gates, flip-flops and timing
- Top-down: requirements flow from system to logic to circuits

Computer design flows top-down — system requirements shape logic design, which shapes circuit design.

Design considerations

- At every level, computer design balances competing goals
- **Performance:** how fast the resulting hardware runs
- **Cost:** how expensive it is to manufacture, which often trades against performance
- **Compatibility:** whether the design correctly implements the required architecture, so existing software keeps working

Unit 1 summary (1 of 2)

- **1.1 Number systems:** positional notation with radix r; repeated division or multiplication to convert; group bits for octal and hex
- **1.2 Complements:** (r − 1)'s and r's complements let one adder subtract; the end carry decides the sign
- **1.3 Fixed-point:** a sign bit; signed-2's-complement adds with one rule; overflow when the carries into and out of the sign bit differ
- **1.4 Floating-point:** mantissa × 2^{exponent}; normalise the mantissa; zero is all zeros
- **1.5 Binary codes:** BCD, Excess-3, Gray code (one bit changes), ASCII and parity
- **1.6 Storage and registers:** a flip-flop stores one bit; n cells hold 2^n values
- **1.7 Binary logic:** AND, OR and NOT on truth values

Unit 1 summary (2 of 2)

- **1.8 Logic gates:** seven basic gates, DeMorgan's theorem, NAND and NOR are universal, canonical SOP
- **1.9 K-maps:** Gray-code headers; group adjacent 1s in powers of two to simplify
- **1.10 Digital computers:** binary digits; the same bits mean different things under different codes; hardware and software
- **1.11 Block diagram:** CPU (ALU, registers, control), memory (RAM) and the input-output processor
- **1.12 Organization vs architecture:** architecture is what the programmer sees; organization is how the hardware works
- **1.13 Computer design:** system, logic and circuit design, balancing performance, cost and compatibility
- **Key applications:** 2's complement in every CPU; hex for addresses, colour codes and networking; Gray code in rotary encoders